Rockchip RT-Thread SPI2APB Developer Guide

ID: RK-KF-YF-498

Release Version: V2.2.0

Release Date: 2025-11-20

Security Level: ☐Top-Secret ☐Secret ☐Internal ■Public

DISCLAIMER

THIS DOCUMENT IS PROVIDED “AS IS”. ROCKCHIP ELECTRONICS CO., LTD.(“ROCKCHIP”)DOES NOT PROVIDE ANY WARRANTY OF ANY KIND, EXPRESSED, IMPLIED OR OTHERWISE, WITH RESPECT TO THE ACCURACY, RELIABILITY, COMPLETENESS,MERCHANTABILITY, FITNESS FOR ANY PARTICULAR PURPOSE OR NON-INFRINGEMENT OF ANY REPRESENTATION, INFORMATION AND CONTENT IN THIS DOCUMENT. THIS DOCUMENT IS FOR REFERENCE ONLY. THIS DOCUMENT MAY BE UPDATED OR CHANGED WITHOUT ANY NOTICE AT ANY TIME DUE TO THE UPGRADES OF THE PRODUCT OR ANY OTHER REASONS.

Trademark Statement

"Rockchip", "瑞芯微", "瑞芯" shall be Rockchip’s registered trademarks and owned by Rockchip. All the other trademarks or registered trademarks mentioned in this document shall be owned by their respective owners.

All rights reserved. ©2025. Rockchip Electronics Co., Ltd.

Beyond the scope of fair use, neither any entity nor individual shall extract, copy, or distribute this document in any form in whole or in part without the written approval of Rockchip.

Rockchip Electronics Co., Ltd.

No.18 Building, A District, No.89, software Boulevard Fuzhou, Fujian,PRC

Website: www.rock-chips.com

Customer service Tel: +86-4007-700-590

Customer service Fax: +86-591-83951833

Customer service e-Mail: fae@rock-chips.com

Preface

Overview

This document primarily describes the usage method of the ROCKCHIP RT-Thread SPI2APB driver.

Product Version

Chipset	Kernel Version
All chip products using RK RT-Thread SDK	RT-Thread

Intended Audience

This document (this guide) is mainly intended for:

Technical support engineers

Software development engineers

Revision History

Version	Author	Date	Modification Description
V1.0.0	Lin Dingqiang	2024-03-28	Initial version
V1.1.0	Lin Dingqiang	2024-04-26	Added common questions
V1.2.0	Lin Dingqiang	2024-05-25	Added flash_obj application support
V2.0.0	Lin Dingqiang	2025-07-12	Added support for RK2116
V2.1.0	Lin Dingqiang	2025-07-29	Updated SPI parameter configuration
V2.2.0	Lin Dingqiang	2025-11-20	Add RT-Thread host driver, Reserved memory usage, remove flash_obj

Contents

Rockchip RT-Thread SPI2APB Developer Guide

1. Introduction
   - 1.1 Notice
   - 1.2 Controller Features
     - 1.2.1 V2 Features
     - 1.2.2 V4 Features
   - 1.3 SPI2APB Protocol
     - 1.3.1 V2 Protocol
     - 1.3.2 V4 Protocol
   - 1.4 Interface Features
2. Software
   - 2.1 Code Path
   - 2.2 Macro Switch Configuration
   - 2.3 SPI Parameter Configuration
   - 2.4 Dedicated Reserved Memory
3. Testing
   - 3.1 Configuration
   - 3.2 Test Commands
4. Testing
   - 4.1 Configuration
   - 4.2 Test Commands
5. Host Driver Introduction
   - 5.1 Linux Deivce Driver
   - 5.2 RT-Thread Host Test Driver
6. Common Issues
   - 6.1 SPI2APB Transfer Failure

1. Introduction

1.1 Notice

The SPI2APB controller superimposes RK proprietary protocols on top of the standard SPI communication protocol, which is different from the more universal RK SPI module without protocol superimposition. It is recommended that customers pay attention to this situation when reviewing the schematics.

1.2 Controller Features

1.2.1 V2 Features

Refer to "Interface Features" for information on the controller IP version. The V2 version controller supports the following features:

- Supports 2 types of SPI modes, SPI mode 0/2, with mode 0 recommended
- Supports 8-bit transmission
- Supports various interrupt types:
  - Feature protocol write REG1 triggers SPI2APB interrupt
  - Supports CS extended GPIO interrupt
- Supports reading and writing to controller internal registers
- Supports configurable LSB/MSB
- Supports configurable Little Endian/Big Endian
- Only supports simplex transmission
- Only supports slave mode

1.2.2 V4 Features

Refer to "Interface Features" to confirm the controller IP version information. The V4 version controller supports the following features:

- Supports 4 types of SPI modes, with mode 0 recommended
- Supports 8bits data transfer
- Supports various interrupt types:
  - Feature protocol write REG1 triggers SPI2APB interrupt
  - CS interrupt
  - Transfer behavior interrupt
  - Transfer exception interrupt
- Supports reading and writing to the controller's internal registers

- Supports configurable LSB/MSB
- Supports configurable Little Endian/Big Endian
- Only supports simplex transmission
- Only supports slave mode
- Supports pass-through mode

1.3 SPI2APB Protocol

SPI master should adhere to the SPI2APB protocol to initiate the transmission of specific data. The basic framework of the protocol is `<CMD-32bits> [ADDR-32bits] [DUMMY-32bits] [READ_BEGING-32bits] [DATA]`.

1.3.1 V2 Protocol

Refer to "Interface Features" to confirm the controller IP version information. The V2 protocol supports the following RK custom transmission protocols:

Description	Protocol Abbreviation	Protocol (CMD-32bits Little-Endian Transmission)	SPI2APB Side Driver Configuration
Read Data	read data	`<0x00000077> <ADDR> <DUMMY> <READ_BEGING> <DATA>`	
Write Data	write data	`<0x00000011> <ADDR> <DATA>`	
Write Controller Internal REG0 Register	write msg reg0	`<0x00010011> <DATA-32bits>`	Read reg0 value on your own
Write Controller Internal REG1 Register, Trigger Interrupt	write msg reg1	`<0x00020011> <DATA-32bits>`	Register and handle ISR Callback
Query Previous Transmission Status	query state	`<0x000000ff> <DATA-32bits>`	

Description	Protocol Abbreviation	Protocol (CMD-32bits Little-Endian Transmission)	SPI2APB Side Driver Configuration
Read Controller Internal REG2 Register	query msg reg2	`<0x000001ff> <DATA-32bits>`	Pre-fill reg2 value

Explanation:

- [ADDR-32bits] refers to the memory address supported by SPI2APB, typically sram/dram.
- [DUMMY-32bits] has no requirement and can be filled with 0 directly.
- [READ_BEGING-32bits] value is 0x000000AA.
- The return value of query state refers to the "State Register Information" section in the TRM SPI2APB chapter.

1.3.2 V4 Protocol

Refer to "Interface Features" to confirm the controller IP version information. The V4 protocol is backward compatible with all V2 protocols and adds the following support:

- New RK custom transmission protocol:

Description	Protocol Abbreviation	Protocol (CMD-32bits Little Endian Transmission)	SPI2APB Side Driver Configuration
Read Data Bypass ADDR	read data bypass addr	`<0x00008077> <DUMMY> <READ_BEGING> <DATA>`	DST_ADDR sets the target address
Write Data Bypass ADDR	write data bypass addr	`<0x00008011> <DATA>`	DST_ADDR sets the target address
Write Controller Internal REG3 Register	write msg reg3	`<0x00040011> <DATA-32bits>`	
Read Controller Internal REG4 Register	query msg reg4	`<0x000002ff> <DATA-32bits>`	

- New simulation of ordinary SPI protocol functionality:

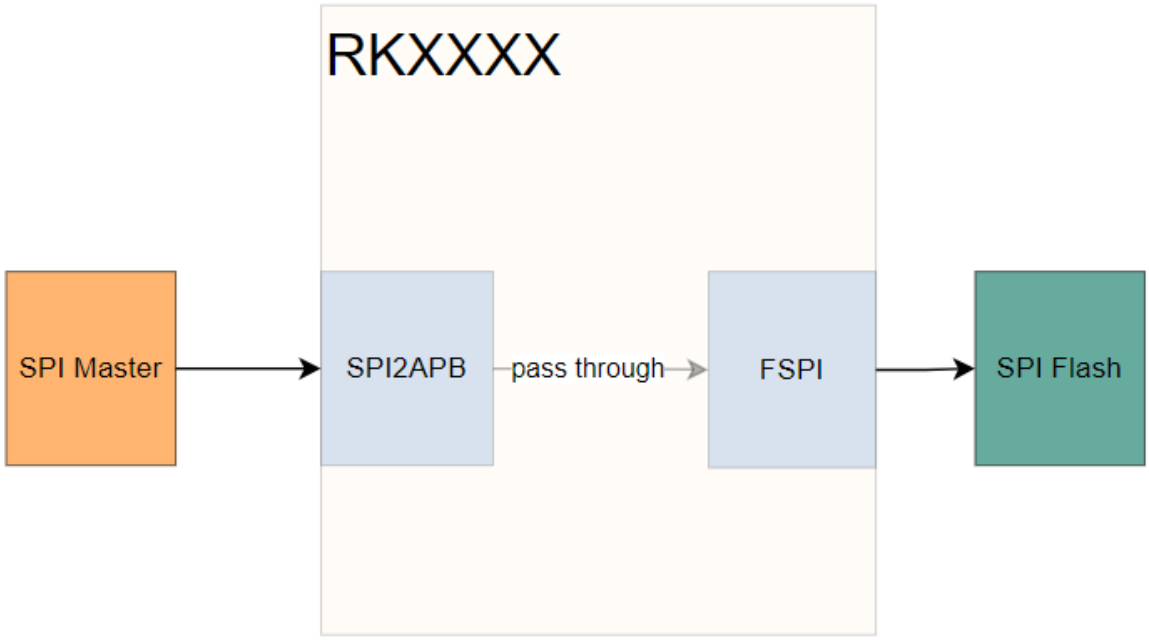
Description	Protocol Abbreviation	Protocol (CMD-32bits Little Endian Transmission)	SPI2APB Required Configuration
SPI Unidirectional Write (No Protocol)	read data bypass ca	No protocol	CTRL0 sets write behavior DST_ADDR sets the target address
SPI Unidirectional Read (No Protocol)	write data bypass ca	No protocol	CTRL0 sets read behavior DST_ADDR sets the target address

- New Direct Pass-through Mode:

Description	Protocol Abbreviation	Protocol (CMD-32bits Little Endian Transmission)	SPI2APB Required Configuration
Hardware Enters Direct Pass-through Mode	Enter TMD By HW	0x00de017b	
Hardware Exits Direct Pass-through Mode	Exit TMD By HW	0x0000017b	
Software Enters Direct Pass-through Mode	Enter TMD By SW	0x00de027b	ISR configures TMC->STMC 1
Software Exits Direct Pass-through Mode	Exit TMD By SW	0x0000027b	ISR configures TMC->STMC 0

Explanation:

- SPI2APB provides Through mode, supporting direct access to off-chip host and off-chip flash operations, while also supporting hardware self-switching and software configuration methods to enter and exit Through mode. As shown in the figure below, when SPI2APB receives a specific command to enter Through mode, SPI Master read and write data will pass directly to SPI Flash.



1.4 Interface Features

SOC	Maximum Interface Speed	IP Version	Support for Through Mode to FSPI, Recommended Speed
RK2118	50MHz	V2	N
RK2116	50MHz	V4	SPI2APB0 (SPI0) Y, within 15MHz SPI2APB1 (SPI3) N

Explanation:

- The maximum interface speed is theoretical and is subject to the quality of device wiring and PCB, with actual measurements being the standard.
- SPI2APB data transmission length requires 4-byte alignment; other transmissions have no alignment restrictions.
- Due to internal path delays in Through Mode, it is recommended to reduce the interface speed to within the recommended speed. If the SPI Master supports adjustable sampling delays, it can be compatible with the maximum speed.

2. Software

2.1 Code Path

Framework Code:

```
components/drivers/include/drivers/spi.h
components/drivers/spi/spi_core.c
components/drivers/spi/spi_dev.c
```

Driver Adaptation Layer:

```
bsp/rockchip/common/drivers/drv_spi2apb.c
bsp/rockchip/common/hal/lib/hal/src/hal_spi2apb.c
```

Test Command, user programs can fully refer to the following driver:

```
bsp/rockchip-common/tests/spi2apb_test.c
```

2.2 Macro Switch Configuration

- Enable the option labeled with SPI2APB

```
RT-Thread bsp drivers  -->
    RT-Thread rockchip "project" drivers  -->
        [*] Enable SPI
        [*]   Enable SPI0 (SPI2APB)
        [ ]   Enable SPI1
        [ ]   Enable SPI2
```

- Configure iomux at the board level.

2.3 SPI Parameter Configuration

After the SPI2APB controller is initialized and before calling rt_spi_configure, the default SPI2APB configuration is as follows:

- MSB, Big Endian, mode 0
- The default configuration is usually not directly usable. The main reason for choosing this set of configurations is to prevent SPI2APB from receiving redundant data packets before the application driver is initialized.

SPI2APB is registered as a standard RT-Thread SPI device, supporting modification of SPI parameter configurations through the rt_spi_configure interface:

- Supports modification of mode, V2 version only supports mode 0/2, other versions support mode0/1/2/3, mode 0 is recommended
- Supports modification of MSB/LSB, LSB is recommended
- Supports modification of Big Endian/Little Endian, Little Endian is recommended

2.4 Dedicated Reserved Memory

Some platforms reserve dedicated memory for SPI2APB by default to cache data, typically with a size of 4KB. After compiling the system firmware, details can be obtained using the following keywords. Take RK2116 as an example:

```
grep -rn __spi2apb_buffer_start__ rtt.map
12396:                0x001a1000                        PROVIDE
(__spi2apb_buffer_start__ = SRAM_TRANS_BASE)
grep -rn SRAM_TRANS_SIZE rt.map
rtt.map:7176:                0x00001000                        SRAM_TRANS_SIZE =
0x1000
```

Notes:

- In the above example, the address is 0x001a1000 and the size is 0x1000
- For how to use this address, refer to the definition of SRAM_SPI2APB_BASE in the tests/spi2apb_test.c source code

3. Testing

3.1 Configuration

```
RT-Thread bsp test case  --->
    [*] RT-Thread Common Test case  --->
        [] Enable BSP Common TEST
        [*]  Enable BSP Common SPI2APB TEST
```

3.2 Test Commands

Source Code:

```
bsp/rockchip-common/tests/spi2apb_test.c
```

Commands:

```
spi2apb_test config spi2apb 0 0         # <mode> <is_msb>
spi2apb_test cb spi2apb                 # Register SPI2APB->REG1 interrupt
callback
spi2apb_test read spi2apb 0             # Read SPI2APB->REG0 info, recommended
debug method: master device first writes following 'write msg reg0' protocol,
then slave device executes this read command
spi2apb_test read spi2apb 1             # Read SPI2APB->REG1 info, recommended
debug method: master device first writes following 'write msg reg1' protocol,
then slave device executes this read command
spi2apb_test query spi2apb              # Read SPI2APB->SR info
spi2apb_test write spi2apb 0x12345678   # Write to SPI2APB->REG2 info,
recommended debug method: slave device first executes this command, then master
device reads following 'query msg reg2' protocol
```

Command Descriptions:

- Master device - SPI master, Slave device - RK SPI2APB slave

- config command:
  - mode: V2 version only supports mode 0/2, other versions support mode 0/1/2/3, mode 0 is recommended
  - is_msb: 0-LSB, 1-MSB
  - There is also a configuration RK_SPI_RESERVED_LITTLE_ENDIAN in the source code, which can be modified to configure Big Endian

4. Testing

4.1 Configuration

```
RT-Thread BSP Test Case --->
    [*] RT-Thread Common Test Case --->
        [*] Enable BSP Common TEST
        [*] Enable BSP Common SPI2APB TEST
```

4.2 Test Commands

Source Code:

```
bsp/rockchip-common/tests/spi2apb_test.c
```

Commands:

```
spi2apb_test config spi2apb 0 0         # <mode> <is_msb>
spi2apb_test cb spi2apb                 # Register SPI2APB->REG1 interrupt
callback
spi2apb_test read spi2apb 0             # Read SPI2APB->REG0 information,
recommended debugging method, first write on the master device side according to
'write msg reg0' protocol, then execute this command on the slave device side to
read
spi2apb_test read spi2apb 1             # Read SPI2APB->REG1 information,
recommended debugging method, first write on the master side according to 'write
msg reg1' protocol, then execute this command on the slave side to read
spi2apb_test query spi2apb              # Read SPI2APB->SR information
spi2apb_test write spi2apb 0x12345678   # Write SPI2APB->REG2 information,
recommended debugging method, first execute this command on the slave side, then
read on the master side according to 'query msg reg2' protocol
```

Command Description:

- Master device side - SPI master, slave side - RK SPI2APB slave
- config command:
  - mode, V2 version only supports mode 0/2, other versions support mode0/1/2/3, mode 0 is recommended
  - is_msb: 0-LSB, 1-MSB

- There is also a configuration for RK_SPI_RESERVED_LITTLE_ENDIAN in the source code, which can be modified to configure Big Endian

5. Host Driver Introduction

SPI2APB application requires the master and slave ends to build data streams and control information based on the SPI2APB protocol principles.

5.1 Linux Deivce Driver

Source code acquisition: SPI Development Materials - FAE Project - Rockchip Redmine (rock-chips.com)

Taking the connection of RV1106 SPI master with RK2118 SPI2APB as an example, understand the application implementation from the source code:

Hierarchy Level	Master Device (Linux 5.10)	Slave Device (HAL + RTOS)
User-Level Application	spidev_rkspi2apb_app C program, calling ioctl/read/write interfaces of spi2apb device node	test_spi2apb.c program, obtaining REG register information
Kernel-Level Device Driver	spidev_rkspi2apb.c source code, still implements ioctl/read/write interfaces according to spi2apb protocol	
Controller Driver	spi-rockchip.c source code	drv_spi2apb.c hal_spi2apb.c
Controller	spi master	spi2apb slave

Explanation:

- spidev_rkspi2apb.c ioctl interface description:

```
enum spidev_rkspi2apb_mode {
    SPI2APB_MODE_NORMAL, /* Read and write transmission using write data/read
data mode */
    SPI2APB_MODE_BYPASS_ADDR, /* Read and write transmission using write data
bypass addr/read data bypass addr mode */
    SPI2APB_MODE_BYPASS_CA, /* Read and write transmission using write data
bypass cmd_addr/read data bypass cmd_addr mode, simulating general SPI */
    SPI2APB_MODE_MAX,
};

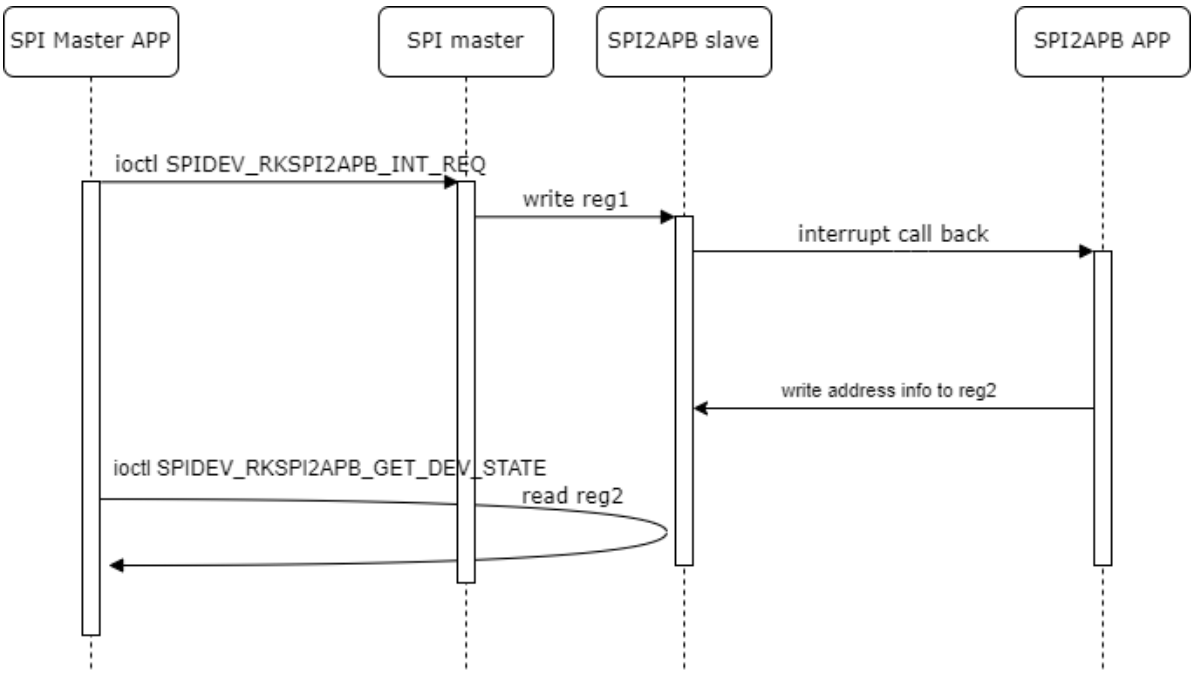
#define SPIDEV_RKSPI2APB_BASE   'S'
#define SPIDEV_RKSPI2APB_GET_DEV_STATE          _IOR(SPIDEV_RKSPI2APB_BASE, 0,
uint32_t) /* Refer to 'query msg reg2' protocol */
#define SPIDEV_RKSPI2APB_SET_DST_ADDR           _IOW(SPIDEV_RKSPI2APB_BASE, 1,
uint32_t) /* Set the target address for transmission, pointing to the memory
address of the slave device */
#define SPIDEV_RKSPI2APB_SEND_MSG               _IOW(SPIDEV_RKSPI2APB_BASE, 2,
uint32_t) /* Refer to 'write msg reg0' protocol */
#define SPIDEV_RKSPI2APB_INT_REQ                _IOW(SPIDEV_RKSPI2APB_BASE, 3,
uint32_t) /* Refer to 'write msg reg1' protocol */

/* VER=0x24060004 Extension */
#define SPIDEV_RKSPI2APB_QUERY_MSG_EXT      _IOR(SPIDEV_RKSPI2APB_BASE, 0x10,
uint32_t) /* Refer to 'query msg reg5' protocol */
#define SPIDEV_RKSPI2APB_SEND_MSG_EXT       _IOW(SPIDEV_RKSPI2APB_BASE, 0x12,
uint32_t) /* Refer to 'write msg reg4' protocol */
#define SPIDEV_RKSPI2APB_USING_MODE     _IOW(SPIDEV_RKSPI2APB_BASE, 0x13, enum
spidev_rkspi2apb_mode) /* Switch read and write transmission modes */
#define SPIDEV_RKSPI2APB_QUERY_STATE        _IOR(SPIDEV_RKSPI2APB_BASE, 0x14,
uint32_t)
```

- spidev_rkspi2apb.c read and write explanation:

```
mode = SPI2APB_MODE_NORMAL;
ioctl(spidev_fd, SPIDEV_RKSPI2APB_USING_MODE, &mode);
ioctl(spidev_fd, SPIDEV_RKSPI2APB_SET_DST_ADDR, &dst_addr) /* Set the target
address for write transmission */
write(spidev_fd, buffer_w, size);
ioctl(spidev_fd, SPIDEV_RKSPI2APB_SET_DST_ADDR, &dst_addr) /* Set the target
address for read transmission */
read(spidev_fd, buffer_r, size);
```

- How to agree on the target memory address for SPI2APB transmission:
    - Pre-set (recommended solution), usually a known target sram address is reserved on the slave device for transmission, and the master device uses ioctl SPIDEV_RKSPI2APB_SET_DST_ADDR to set it. Typically, the RT-Thread solution reserves this memory through the macro __spi2apb_buffer_start__ in the connection script.
    - Custom, the slave device applies for a sram address space, and both ends customize the application, ultimately transmitting the address information to the master device, such as the following reference process:



5.2 RT-Thread Host Test Driver

Code Path:

```
bsp/rockchip/common/tests/spi2apb_mst.c
```

Macro Switch Configuration:

```
RT-Thread Common Test case  --->
[*] Enable BSP Common TEST
[*]   Enable BSP Common SPI2APB Master TEST (NEW)
```

Corresponding Macro:

```
RT_USING_COMMON_TEST_SPI2APB=y
```

Interface Introduction:

```
spi2apb_mst write <spi_dev> <addr> <size> <loops> <val> # Refer to "write data"
protocol
spi2apb_mst read <spi_dev> <addr> <size> <loops> # Refer to "read data" protocol
```

SPI Configuration Modification: defaults to lsb, mode0, 25MHz, where lsb and mode0 should be consistent with SPI2APB:

```
spi2apb_mst_init()
{
    cfg.data_width = 8;
    cfg.mode = RT_SPI_MASTER | RT_SPI_MODE_0 | RT_SPI_LSB;
    cfg.max_hz = 25000000;
}
```

6. Common Issues

6.1 SPI2APB Transfer Failure

It is recommended to refer to the "Testing" section to configure the controller in SPI mode 0, lsb, little-endian, and register callback functions with the following commands:

```
spi2apb_test config spi2apb 1 2
spi2apb_test cb spi2apb
```

Explanation:

- The master should refer to the "Data Transfer Protocol" for initiating the transfer, and it is suggested to first attempt "write reg1" to trigger the SPI2APB interrupt. The conventional debugging process includes:
    - Checking if the interconnected devices are grounded together
    - Using an oscilloscope to measure if there are valid signals on the master device's csn/clock
    - Verifying if the slave hardware connections are correct and if there are any 0Ω resistors missing on the circuit
- After the master transfers, if the slave or master receives incorrect data, the conventional debugging process includes:
    - Slowing down the speed
    - Confirming if there is a pattern in the data:
        - Checking if it matches LSB/MSB or little/big-endian configuration issues
        - Checking if only partial valid data is received and if it is related to the cache line size. If so, it is likely that the slave did not add cache consistency operations before and after handling the data
            - Typically, this involves adding cache invalid after read transfers and cache flush before write transfers
            - If the buffer used for read transfers has zeroing or writing operations, it is recommended to add cache flush before the transfer